

SISTEMAS OPERACIONAIS

Prof. Dr. Caetano Mazzoni Ranieri

LISTA DE EXERCÍCIOS - AULA 5

1. Implemente um programa em C que registre um handler para o sinal SIGINT, enviado ao processo quando o usuário pressiona Ctrl+C no terminal.
 - a. Ao receber o sinal, o programa não deve ser encerrado imediatamente. Em vez disso, o handler deve exibir uma mensagem informando que o sinal foi recebido e permitir que o programa continue sua execução.
2. Implemente um pipe em C, como apresentado durante a aula.
3. Escreva um programa contendo múltiplos threads que escrevem repetidamente, **caracter a caracter**, um mesmo conteúdo (e.g., “hello world”) em um buffer compartilhado. A cada iteração do loop, insira uma instrução que faça o programa dormir por um curto período.
 - a. Executando o programa por um período determinado, você verificou alguma condição de corrida?
 - b. Implemente uma solução usando algoritmos de exclusão mútua baseados em espera ocupada e avalie o tamanho do buffer ao final de um determinado período.
 - c. Implemente uma solução usando mutex (lock) ou semáforo.
4. Implemente a simulação de um sistema para processamento de requisições. As requisições são geradas por uma thread despachante, armazenadas em uma fila compartilhada e processadas por uma thread operária. A fila deve armazenar, no máximo, **N** requisições.
 - a. Implemente uma solução usando semáforos e mutexes.
 - b. Implemente uma solução usando o monitor adaptado (ver exemplo). Embora não exista um tipo Monitor na biblioteca padrão do Python, a classe `threading.Condition`, usada em associação com uma instância de `threading.Lock()` permite uma implementação equivalente (ver exemplo).

- c. Modifique a simulação para múltiplas threads operárias.
5. **[Jantar dos filósofos]** Implemente uma simulação do problema do Jantar dos Filósofos. Cinco filósofos estão sentados ao redor de uma mesa e alternam entre os estados de **pensar** e de **comer**. Conforme ilustrado na Figura 1, há um garfo entre cada par de filósofos, e cada filósofo precisa obter os dois garfos adjacentes antes de começar a comer.

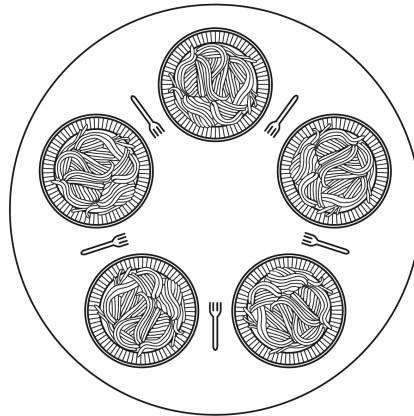


Figura 1. Hora do almoço no departamento de Filosofia.

- a. Implemente cada filósofo como uma thread e cada garfo como um recurso compartilhado protegido por um mutex. Inicialmente, faça com que cada filósofo tente obter primeiro o garfo à sua esquerda e, em seguida, o à direita. Lembre-se de fazer um `time.sleep()` de curta duração entre estas duas ações.
- b. Execute a simulação diversas vezes e verifique se pode ocorrer *deadlock*. Em seguida, modifique a implementação para evitar o *deadlock*.
6. **[Leitores e escritores]** Implemente uma simulação do problema dos Leitores e Escritores. Considere um conjunto de threads que acessam um mesmo recurso compartilhado (e.g., um arquivo ou banco de dados). Existem dois tipos de threads: (i) **leitores**: apenas consultam o recurso e, portanto, vários leitores podem acessá-lo simultaneamente; e (ii) **escritores**: modificam o recurso e precisam de acesso exclusivo. Enquanto um escritor estiver escrevendo, nenhum leitor ou outro escritor pode acessar o recurso.
- a. Implemente uma solução que respeite essas regras de acesso. Cada leitor e cada escritor deve ser implementado como uma thread.

- b. Execute a simulação com vários leitores e poucos escritores. Observe que, se novos leitores puderem entrar enquanto um escritor aguarda, o escritor pode permanecer esperando indefinidamente (*starvation*).
- c. Modifique a implementação para evitar a *starvation* dos escritores.